

# Pro .NET Memory Management

For Better Code, Performance, and Scalability

—

*Second Edition*

—

Konrad Kokosa
Christophe Nasarre
Kevin Gosse

Apress®

# Pro .NET Memory Management

## For Better Code, Performance, and Scalability

Second Edition



Konrad Kokosa
Christophe Nasarre
Kevin Gosse

Apress®

*Pro .NET Memory Management: For Better Code, Performance, and Scalability, Second Edition*

Konrad Kokosa
Warsaw, Poland

Christophe Nasarre
Paris, France

Kevin Gosse
Paris, France

ISBN-13 (pbk): 979-8-8688-0452-6
ISBN-13 (electronic): 979-8-8688-0453-3
https://doi.org/10.1007/979-8-8688-0453-3

Copyright © 2024 by Konrad Kokosa, Christophe Nasarre, and Kevin Gosse

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

Trademarked names, logos, and images may appear in this book. Rather than use a trademark symbol with every occurrence of a trademarked name, logo, or image we use the names, logos, and images only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Managing Director, Apress Media LLC: Welmoed Spahr
Acquisitions Editor: Ryan Byrnes
Development Editor: Laura Berendson
Editorial Project Manager: Jessica Vakili

Cover designed by eStudioCalamar

Distributed to the book trade worldwide by Springer Science+Business Media New York, 1 New York Plaza, Suite 4600, New York, NY 10004-1562, USA. Phone 1-800-SPRINGER, fax (201) 348-4505, e-mail orders-ny@springer-sbm.com, or visit www.springeronline.com. Apress Media, LLC is a California LLC and the sole member (owner) is Springer Science + Business Media Finance Inc (SSBM Finance Inc). SSBM Finance Inc is a **Delaware** corporation.

For information on translations, please e-mail booktranslations@springernature.com; for reprint, paperback, or audio rights, please e-mail bookpermissions@springernature.com.

Apress titles may be purchased in bulk for academic, corporate, or promotional use. eBook versions and licenses are also available for most titles. For more information, reference our Print and eBook Bulk Sales web page at http://www.apress.com/bulk-sales.

Any source code or other supplementary material referenced by the author in this book is available to readers on GitHub. For more detailed information, please visit https://www.apress.com/gp/services/source-code.

If disposing of this product, please recycle the paper

*Kevin Gosse: To my beloved wife, who would be upset if she weren't first on this list.*
*To my son, who kindly napped long enough for me to write.*
*To my parents, who made me who I am.*

*Konrad Kokosa: To my beloved wife, Justyna, without whom*
*nothing really valuable would happen in my life.*

*Christophe Nasarre: To my wife and my dad.*

# Table of Contents

**About the Authors** ..... xvii

**About the Technical Reviewer** ..... xix

**Acknowledgments** ..... xxi

**Foreword** ..... xxiii

**Introduction** ..... xxv

**Chapter 1: Basic Concepts** ..... 1

Memory-Related Terms ..... 2

Static Allocation ..... 4

The Register Machine ..... 5

The Stack ..... 6

The Stack Machine ..... 9

The Pointer ..... 11

The Heap ..... 13

Manual Memory Management ..... 15

Automatic Memory Management ..... 19

Allocator, Mutator, and Collector ..... 20

Reference Counting ..... 23

Tracking Collector ..... 27

Mark Phase ..... 28

Summary ..... 35

Rule 1 – Educate Yourself ..... 35

**Chapter 2: Low-Level Memory Management ..... 37**

Hardware ..... 37

Memory ..... 43

CPU ..... 45

Operating System ..... 59

Virtual Memory ..... 60

Large Pages ..... 63

Virtual Memory Fragmentation ..... 63

General Memory Layout ..... 63

Windows Memory Management ..... 65

Windows Memory Layout ..... 68

Linux Memory Management ..... 70

Linux Memory Layout ..... 72

Operating System Influence ..... 73

NUMA and CPU Groups ..... 73

Summary ..... 75

Rule 2 – Random Access Should Be Avoided, Sequential Access Should Be Encouraged ..... 75

Rule 3 – Improve Spatial and Temporal Data Locality ..... 75

Rule 4 – Consider More Advanced Possibilities ..... 76

**Chapter 3: Memory Measurements ..... 77**

Measure Early ..... 78

Overhead and Invasiveness ..... 78

Sampling vs. Tracing ..... 79

Call Tree ..... 79

Object Graphs ..... 80

Statistics ..... 81

Latency vs. Throughput ..... 85

Memory Dumps, Tracing, Live Debugging ..... 86

Windows and Linux Environments ..... 87

Overview ..... 87

VMMap ..... 87

.NET Framework Performance Counters ..... 88
.NET Core Counters ..... 95
Event Tracing for Windows ..... 98
PerfView ..... 105
dotnet-trace CLI Tool ..... 116
dotnet-gcmon CLI tool ..... 117
ProcDump, dotnet-dump ..... 118
dotnet-gcdump CLI tool ..... 119
WinDbg ..... 120
dotnet-dump for Analysis ..... 123
Disassemblers and Decompilers ..... 123
BenchmarkDotNet ..... 124
Tools from the Authors ..... 125
Commercial Tools ..... 127
Summary ..... 128
Rule 5 – Measure GC Early ..... 130
**Chapter 4: .NET Fundamentals ..... 131**
.NET Versions ..... 131
.NET Internals ..... 133
Sample Program in Depth ..... 136
Assemblies and Application Domains ..... 142
Collectible Assemblies ..... 143
Process Memory Regions ..... 144
Scenario 4-1 – How Big Is My Program in Memory? ..... 149
Scenario 4-2 – My Program's Native Memory Usage Keeps Growing ..... 151
Scenario 4-3 – My Program's Virtual Memory Usage Keeps Growing ..... 154
Scenario 4-4 – My Program's Managed Memory Usage Keeps Growing with Assemblies Count ..... 156
Scenario 4-5 – My Program Is Unable to Unload Plugins ..... 159
Type System ..... 162
Type Categories ..... 163
Type Storage ..... 164

Value Types ..... 165
Reference Types ..... 172
Strings ..... 179
String Interning ..... 183
Scenario 4-6 – My Program's Memory Usage Is Too Big ..... 187
Boxing and Unboxing ..... 190
Passing by Reference ..... 192
Pass-by-Reference Value Type Instance ..... 192
Pass-by-Reference Reference Type Instance ..... 193
Null ..... 194
Type Data Locality ..... 196
Static Data ..... 198
Static Fields ..... 199
Static Data Internals ..... 200
Summary ..... 203
Structs ..... 204
Classes ..... 204
**Chapter 5: Memory Partitioning ..... 207**
Partitioning Strategies ..... 207
Size Partitioning ..... 209
Small Object Heap ..... 210
Large Object Heap ..... 210
Lifetime Partitioning ..... 212
Scenario 5-1 – Is My Program Healthy? Generation Sizes in Time ..... 217
Remembered Sets ..... 222
Card Tables ..... 226
Card Bundles ..... 231
Kind Partitioning ..... 233
The NonGC Heap ..... 233
The Pinned Object Heap ..... 233
Pinned Object Heap and Internal CLR Data ..... 234

Physical Partitioning ..... 236
Segment Implementation (Pre-.NET 7) ..... 237
Region Implementation (.NET 7+) ..... 244
NonGC Heap ..... 248
Scenario 5-2 – nopCommerce Memory Leak? ..... 248
Scenario 5-3 – Large Object Heap Waste? ..... 255
Segments, Regions, and Heap Anatomy ..... 256
Segment Reuse ..... 260
Region Reuse ..... 263
Summary ..... 263
Rule 11 – Monitor Generation Sizes ..... 264
Rule 12 – Avoid Unnecessary Heap References ..... 264
Rule 13 – Monitor Segment Usage ..... 265
**Chapter 6: Memory Allocation ..... 267**
Allocation Introduction ..... 267
Bump Pointer Allocation ..... 268
Free-List Allocation ..... 274
Creating a New Object ..... 279
Small Object Heap Allocation ..... 281
Large Object Heap and Pinned Object Heap Allocation ..... 284
Heap Balancing ..... 287
Pinned Object Heap Allocation API ..... 289
OutOfMemoryException ..... 289
Scenario 6-1 – Out of Memory ..... 290
Stack Allocation ..... 292
Avoiding Allocations ..... 294
Explicit Allocations of Reference Types ..... 295
Creating Streams – Use RecyclableMemoryStream ..... 306
Creating a Lot of Objects – Use Object Pool ..... 309
Async Methods Returning Task – Use ValueTask ..... 311

Hidden Allocations ..... 317
String Encoding ..... 330
Various Hidden Allocations Inside Libraries ..... 333
Scenario 6-2 – Investigating Allocations ..... 337
Scenario 6-3 – Azure Functions ..... 340
Summary ..... 341
Rule 14 – Avoid Allocations on the Heap in Hot Paths ..... 341
Rule 15 – Avoid Excessive UOH Allocations ..... 342
Rule 16 – Allocate on the Stack When Appropriate ..... 342
**Chapter 7: Garbage Collection – Introduction ..... 343**
High-Level View ..... 343
GC Work by Examples – Segments ..... 345
GC Work by Examples – Regions ..... 351
GC Step by Step ..... 352
Scenario 7-1 – Analyzing the GC Usage ..... 353
Profiling the GC ..... 357
Garbage Collection Performance Tuning Data ..... 359
Static Data ..... 359
Dynamic Data ..... 361
Scenario 7-2 – Understanding the Allocation Budget ..... 364
Collection Triggers ..... 372
Allocation Trigger ..... 372
Explicit Trigger ..... 373
Scenario 7-3 – Analyzing the Explicit GC Calls ..... 376
Low Memory Level System Trigger ..... 382
Various Internal Triggers ..... 383
EE Suspension ..... 384
Scenario 7-4 – Analyzing GC Suspension Times ..... 386
Collection Tuning ..... 388
Hard Memory Limit ..... 392

Provisional Mode ..... 392
"Servo Tuning" ..... 393
Scenario 7-5 – Condemned Generation Analysis ..... 393
Scenario 7-6 – Provisional Mode ..... 395
Summary ..... 397
**Chapter 8: Garbage Collection – Mark Phase ..... 399**
Object Traversal and Marking ..... 399
Local Variable Roots ..... 401
Local Variable Storage ..... 401
Stack Roots ..... 402
Lexical Scope ..... 402
Live Stack Roots vs. Lexical Scope ..... 403
Live Stack Roots with Eager Root Collection ..... 404
GC Info ..... 409
Pinned Local Variables ..... 414
Stack Root Scanning ..... 416
Finalization Roots ..... 417
GC Internal Roots ..... 417
GC Handle Roots ..... 418
Handling Memory Leaks ..... 424
Scenario 8-1 – nopCommerce Memory Leak? ..... 426
Scenario 8-2 – Identifying the Most Popular Roots ..... 430
Scenario 8-3 – Generational Aware Analysis ..... 432
Summary ..... 435
**Chapter 9: Garbage Collection – Plan Phase ..... 437**
Small Object Heap ..... 437
Plugs and Gaps ..... 438
Scenario 9-1 – Memory Dump with Invalid Structures ..... 442
Brick Table ..... 443
Pinning ..... 445

Scenario 9-2 – Investigating Pinning ..... 450
Generation Boundaries ..... 454
Demotion ..... 455
Large Object Heap ..... 459
Plugs and Gaps ..... 459
Decide on Compaction ..... 461
Special Regions ..... 462
Summary ..... 463
**Chapter 10: Garbage Collection – Sweep and Compact ..... 465**
Sweep Phase ..... 465
Small Object Heap ..... 465
LOH and POH ..... 466
Compact Phase ..... 467
Small Object Heap ..... 467
Large Object Heap ..... 471
Scenario 10-1 – Large Object Heap Fragmentation ..... 472
Summary ..... 480
Rule 17 – Watch Runtime Suspensions ..... 481
Rule 18 – Avoid Mid-Life Crisis ..... 481
Rule 19 – Avoid Old Generation and LOH Fragmentation ..... 482
Rule 20 – Avoid Explicit GC ..... 482
Rule 21 – Avoid Memory Leaks ..... 483
Rule 22 – Use Pinning Carefully ..... 483
**Chapter 11: GC Flavors and Settings ..... 485**
Mode Overview ..... 485
Workstation vs. Server Mode ..... 485
Non-concurrent vs. Concurrent Mode ..... 487
Mode Configuration ..... 488
.NET Framework ..... 489
.NET Core ..... 489

GC Pause and Overhead ..... 490
Mode Descriptions ..... 492
Workstation Non-concurrent ..... 493
Workstation Concurrent (Before 4.0) ..... 494
Background Workstation ..... 495
Server Non-concurrent ..... 504
Background Server ..... 505
Latency Modes ..... 507
Batch Mode ..... 507
Interactive ..... 508
Low Latency ..... 508
Sustained Low Latency ..... 509
No GC Region ..... 510
Latency Optimization Goals ..... 512
Choosing the GC Flavor ..... 513
Other GC Configuration Knobs ..... 514
Changing the Heap Limits ..... 514
Changing the Number of Heaps ..... 514
Changing the GC Thread Affinity ..... 514
Memory Load Threshold ..... 515
GC Conservative Mode ..... 515
Dynamic Adaptation Mode ..... 516
Large Pages ..... 516
Scenario 11-1. Checking GC Settings ..... 517
Scenario 11-2. Benchmarking Different GC Modes ..... 519
Summary ..... 526
Rule 23 – Choose GC Mode Consciously ..... 527
Rule 24 – Remember About Latency Modes ..... 527

**Chapter 12: Object Lifetime ..... 529**

Object vs. Resource Life Cycle ..... 529

Finalization ..... 531

Introduction ..... 531

Eager Root Collection Problem ..... 534

Critical Finalizers ..... 537

Finalization Internals ..... 538

Scenario 12-1 – Finalization Memory Leak ..... 544

Resurrection ..... 550

Disposable Objects ..... 553

Safe Handles ..... 560

Weak References ..... 564

Caching ..... 569

Weak Event Pattern ..... 571

Scenario 12-2 – Memory Leak Because of Events ..... 576

Summary ..... 579

Rule 25 – Avoid Finalizers ..... 579

Rule 26 – Prefer Explicit Cleanup ..... 580

**Chapter 13: Miscellaneous Topics ..... 581**

Dependent Handles ..... 581

Thread Local Storage ..... 587

Thread Static Fields ..... 587

Thread Data Slots ..... 593

Thread Local Storage Internals ..... 594

Usage Scenarios ..... 601

Managed Pointers ..... 602

Ref Locals ..... 603

Ref Returns ..... 604

Readonly Ref and in Parameters ..... 606

Ref Type Internals ..... 610

Managed Pointers in C# – Ref Variables ..... 620

More on Structs. . . . . 625
Readonly Structs . . . . . 626
Ref Structs and Ref Fields . . . . . 627
Fixed-Size Buffers . . . . . 631
Inline Arrays . . . . . 634
Object/Struct Layout . . . . . 636
Unmanaged Constraint . . . . . 646
Blittable Types . . . . . 650
Summary . . . . . 652
**Chapter 14: Advanced Techniques . . . . . 653**
Span<T> and Memory<T> . . . . . 653
Span<T> . . . . . 653
Memory<T> . . . . . 666
IMemoryOwner<T> . . . . . 668
Memory<T> Internals . . . . . 672
Span<T> and Memory<T> Guidelines . . . . . 674
Unsafe . . . . . 674
Data-Oriented Design . . . . . 678
Tactical Design . . . . . 680
Strategic Design . . . . . 683
Pipelines . . . . . 692
Summary . . . . . 697
**Chapter 15: Programmatical APIs . . . . . 699**
GC API . . . . . 699
Collection Data and Statistics . . . . . 699
GC Notifications . . . . . 711
Controlling Unmanaged Memory Pressure . . . . . 713
Explicit Collection . . . . . 713
No-GC Regions . . . . . 714
Finalization Management . . . . . 714

Memory Usage....714
Memory Limits....716
Internal Calls in the GC Class....717
Frozen Segments....718
ClrMD....720
TraceEvent Library....731
Custom GC....736
Summary....738

**Index....741**

# About the Authors

**Konrad Kokosa** is an experienced software designer and developer with a specific interest in Microsoft technologies, but also looking with curiosity at everything else. He has been programming for over a dozen years, solving performance problems and architectural puzzles in the .NET world, and designing and speeding up .NET applications. He is an independent consultant, blogger, meetup and conference speaker, and fan of X (@konradkokosa). He also shares his passion as a trainer in the area of .NET, especially regarding application performance, coding good practices, and diagnostics. He is a Microsoft MVP in the Developer Technologies category. He is cofounder of the Dotnetos.org initiative of three .NET fans organizing tours and conferences about .NET performance. Also, he is a skeptical fan of Web3 and blockchain technologies. He is recently involved in research and development in the AI area at Nethermind.

**Christophe Nasarre** has been developing and shipping software on Microsoft stacks for 30+ years. He has been working as a technical reviewer since 1996 on books such as *CLR via C#* and the last editions of *Windows Internals*. He provides tools and insights on .NET/Windows development and troubleshooting via X (@chnasarre), and his open source projects are available on GitHub. He is a Microsoft Dev Technologies MVP on top of his job as a software engineer in the Profiling team at Datadog.

**Kevin Gosse** has been using Microsoft .NET technologies since the early days, across client, server, and mobile applications. He is a Microsoft MVP and is currently employed at Datadog, where he works on the performance of the .NET APM. He writes deep-dive technical articles on his personal blog (minidump.net) and is active on Twitter under the alias @kookiz.

# About the Technical Reviewer

**Fabio Claudio Ferracchiati** is a senior consultant and a senior analyst/developer using Microsoft technologies. He works for TIM (www.tim.it). He is a Microsoft Certified Solution Developer for .NET, a Microsoft Certified Application Developer for .NET, a Microsoft Certified Professional, and a prolific author and technical reviewer. Over the past ten years, he's written articles for Italian and international magazines and coauthored more than ten books on a variety of computer topics.

# Acknowledgments

**Konrad Kokosa**

That's the second time I wrote this book! While all original acknowledgments are obviously still valid, and you can read them below, for the second edition I must start by thanking the wonderful coauthors Chris and Kevin with whom I have had the pleasure of coauthoring this edition. The passion and dedication I have seen in them for over the year of working together has boosted me much more than once and reminded me of my own passion. Thank you for the countless hours of discussion on the calls, the hundreds or even thousands of comments, and, most of all, the tremendous substantive input to improve and add new value to the book. Thanks to you, the book now looks the way I wanted it to look from the very beginning. Inviting you was the best decision I could make.

Having said that, let's return to the original acknowledgments.

First of all, I would like to thank my wife very, very much. Without her support, this book would never have been created. Starting to work on this book, I did not imagine how much time not spent together we would have to sacrifice while writing it. Thank you for all the patience, support, and encouragement you have given to me during this time!

Secondly, I would like to thank Maoni Stephens for such extensive, accurate, and invaluable remarks when reviewing the first versions of this book. Without a shadow of a doubt, I can say that thanks to her, this book is better. And the fact that the lead .NET GC developer helped me in writing this book is for me a reward in itself! Many thanks go also to other .NET team members that helped in reviewing some parts of the book, organized also with great help from Maoni (ordered by the amount of work they contributed): Stephen Toub, Jared Parsons, Lee Culver, Josh Free, and Omar Tawfik. I would like also to thank Mark Probst from Xamarin; he reviewed notes about Mono runtime. And special thanks go to Patrick Dussud, "the father of .NET GC," for taking time to review the history of CLR creation.

Thirdly, I would like to thank Damien Foggon, technical reviewer from Apress, who put so much work into a meticulous review of all chapters. His experience in publishing and writing was invaluable to make this book clearer and more consistent. Not once or twice, I was surprised by the accuracy of Damian's comments and suggestions!

I would obviously like to thank everyone at Apress, without whom this book wouldn't have been published in the first place. Special thanks go to Laura Berendson (Development Editor), Nancy Chen (Coordinating Editor), and Joan Murray (Senior Editor) for all the support and patience in extending the deadline again and again. I know there was a time when the date of delivery of the final version was taboo between us! I would also like to thank Gwenan Spearing, with whom I started working on the book, but I did not manage to finish it before she left the Apress team.

I would like to thank a great .NET community in Poland and all around the world, for inspirations from so many great presentations given, articles and posts written by you, for all encouragement and support, and for endless questions about "how a book goes?" Such thanks especially go to (alphabetically): Maciej Aniserowicz, Arkadiusz Benedykt, Sebastian Gębski, Michał Grzegorzewski, Jakub Gutkowski, Paweł Klimczyk, Szymon Kulec, Paweł Łukasik, Alicja Musiał, Łukasz Olbromski, Łukasz Pyrzyk, Bartek Sokół, Sebastian Solnica, Paweł Sroczyński, Jarek Stadnicki, Piotr Stapp, Michał Śliwoń, Szymon Warda, and Artur Wincenciak, all MVP guys (Azure guys, looking at you!), and many more; and I sincerely apologize for those omitted – a big thank you to everyone who feels like receiving such thanks. It is simply not possible to list all of you here. You've inspired me and encouraged me.

I'd like to thank all experienced writers that found time to give me advice about book writing, including Ted Neward (http://blogs.tedneward.com/) and Jon Skeet (https://codeblog.jonskeet.uk) – although I bet they do not remember those conversations! Andrzej Krzywda (http://andrzejonsoftware.blogspot.com) and Gynvael Coldwind (https://gynvael.coldwind.pl) also gave me a lot of very valuable advices on writing and publishing a book.

Next, I'd like to thank all the great tools and library creators that I've used during this book writing: Andrey Shchekin, a creator of SharpLab (https://sharplab.io); Andrey Akinshin, a creator of BenchmarkDotNet (https://benchmarkdotnet.org), and Adam Sitnik, the main maintainer of it; Sergey Teplyakov, a creator of ObjectLayoutInspector (https://github.com/SergeyTeplyakov/ObjectLayoutInspector); 0xd4d, an anonymous creator of dnSpy (https://github.com/0xd4d/dnSpy); Sasha Goldshtein, creator of many useful auxiliary tools (https://github.com/goldshtn); and creators of such great tools like PerfView and WinDbg (and all its .NET-related extensions).

I'd also like to thank my former employer, Bank Millennium, who helped and supported me in starting to write this book. Our path has parted, but I will always remember that it was there that my writing, blogging, and speaking adventure began. Many thanks go also collectively to my former colleagues from there, for the same amount of encouragement and motivation by the "how a book goes?" question.

I'd like to thank all anonymous Twitter users that answered my book-related surveys, giving me directions about what is – and what is not – interesting, useful, and valuable for our .NET family.

And last, but not the least, I would collectively thank all my family and friends that missed me during my work on this book.

**Christophe Nasarre**

Obviously, I want to thank Konrad Kokosa for having asked me to be part of the adventure of writing this second edition with him and Kevin. I'm more than happy to have said *yes*.

Second, I warmly thank my wife for her never-ending support for all these months during which I worked in my ivory tower. It is our time now!

Finally, I need to thank Visual Studio for its great capacity of searching in a large repository, debugging the CLR, and setting data points that greatly helped me to understand how the GC is working and saved me countless hours of investigation.

**Kevin Gosse**

Let's start by thanking my wife once again, because I couldn't have signed this book without her support. Behind any married author stands an understanding wife.

Then of course I would like to thank Konrad for giving me this opportunity and trusting me with the second edition of his precious book. This was unexpected, and a great way to step into the world of book authoring. Though, let's be honest here, I'm really not eager to write another one.

Christophe deserves some thanks too, as he took care of the planning and the communication with Apress, in addition to the actual writing (it turns out that Konrad and I have a strong tendency to, uh, improvise – let's call that "being agile"). He really was instrumental in channeling our scattered efforts.

Lastly, thank you, whoever you are, for reading this! Whether you bought the book, borrowed it, received it as a gift, or just downloaded it from a shady website, the hundreds of hours we dedicated to this would be meaningless without you.

# Foreword

**Second Edition**

The memory space is constantly evolving. Demanding requests are made to memory managers to improve memory usage efficiency. This could mean handling more stressful workloads with less memory, achieving predictable sizes, minimizing interruptions from memory managers, and addressing many other challenges. How we run services has changed quite dramatically. Given the increasing prevalence of containerized environments, there is more pressure for memory managers to be more adaptable to them. Diagnostics also became more challenging for many reasons including difficulty to get to the processes for which you want to investigate memory issues.

.NET runs on a wide range of scenarios, which means we are always making improvements to the garbage collector. Since the first edition of this book, many changes have been made. For example, the physical representation of the GC heap changed from segments to regions which are much smaller units of memory with very different performance characteristics that allow better usage of memory on the heap and innovations that were difficult with segments. The second edition of this book is the only book I'm aware of to date that talks about this important change.

Diagnosing memory issues is no trivial task. Common memory tooling usually analyzes memory snapshots to give you a heap graph or a list of objects with their types and sizes. But memory problems are often not point-in-time issues – you need to look at the history to understand how the heap situation came to be. PerfView, an advanced tool provided by the .NET runtime team, offers a historical view of GCs occurred in your processes with extensive performance information about these GCs. More PerfView usage examples were added to the second edition, which makes it even more useful for diagnosing hard problems.

I think folks who enjoyed the first edition will appreciate the new content in this edition. And for anyone who wants to get a book to understand how the .NET GC works or how to optimize memory usage or diagnose memory issues on .NET, I highly recommend this book.

Maoni Stephens
June 2024

**First Edition**

When I joined the Common Language Runtime (the runtime for .NET) team more than a decade ago, little did I know this component called the Garbage Collector was going to become something I would spend most of my waking moments thinking about later in my life. Among the first few people I worked with on the team was Patrick Dussud, who had been both the architect and dev for the CLR GC since its inception. After observing my work for months, he passed the torch, and I became the second dedicated GC dev for CLR.

And so my GC journey began. I soon discovered how fascinating the world of garbage collection was – I was amazed by the complex and extensive challenges in a GC and loved coming up with efficient solutions for them. As the CLR was used in more scenarios by more users, and memory being one of the most important performance aspects, new challenges in the memory management space kept coming up. When I first started, it was not common to see a GC heap that was even 200 MB; today a 20 GB heap is not uncommon at all. Some of the largest workloads in the world are running on CLR. How to handle memory better for them is no doubt an exciting problem.

In 2015 we open sourced CoreCLR. When this was announced, the community asked whether the GC source would be excluded in the CoreCLR repo – a fair question as our GC included many innovative mechanisms and policies. The answer was a resounding no, and it was the same GC code we used in CLR. This clearly attracted some curious minds. A year later I was delighted to learn that one of our customers was planning to write a book specifically about our GC. When a technology evangelist from our Polish office asked me if I would be available to review Konrad's book, of course I said yes!

As I received chapters from Konrad, it was clear to me that he studied our GC code with great diligence. I was very impressed with the amount of detail covered. Sure, you can build CoreCLR and step through the GC code yourself. But this book will definitely make that easier for you. And since an important class of readers of this book is GC users, Konrad included a lot of material to better understand the GC behavior and coding patterns to use the GC more efficiently. There is also fundamental information on memory at the beginning of the book and discussions of memory usage in various libraries toward the end. I thought it was a perfect balance of GC introduction, internals, and usage.

If you use .NET and care about memory performance, or if you are just curious about the .NET GC and want to understand its inner workings, this is the book to get. I hope you will have as much enjoyment reading it as I did reviewing it.

Maoni Stephens
July 2018

# Introduction

## What's New in the Second Edition

The original book has been completely reviewed both technically and by improving the "style" to provide easier reading. Out-of-date parts have been removed and are now available for free at https://prodotnetmemory.com/. A special effort has been made to present and use the .NET CLI tool set for both Windows and Linux in real-world scenarios. Additional undocumented configuration settings are also explained.

This second edition covers more .NET runtimes including .NET Framework 4.7+, .NET Core 3.0, 3.1, .NET 5 up to 8. Here is a summary of the main additions and enhancements made for this edition:

**Chapter 2**

- Hardware intrinsics and LoadAlignedNonTemporal

**Chapter 3**

- .NET CLI tooling with Linux coverage (dotnet trace, counters, dump, gcmon, gcdump + free content for pre-.NET 3.0 Linux tooling)

**Chapter 4**

- AssemblyLoadContext
- Escape Analysis
- StringBuilder internals
- Pinned Object Heap
- NonGC (or Frozen) Heap usage
- dotnet-counters on Linux for memory usage
- Stack object allocation JIT improvements
- Memory leak investigation step by step

**Chapter 5**

- NonGC heap usage internals
- Pinned Object Heap usage internals
- GC region implementation

**Chapter 6**

- NonGC heap usage
- Pinned Object Heap API and allocation internals
- ObjectPool usage to reduce heap allocations
- Local functions and lambda allocations' specific cases
- Deep coverage of string concatenation optimization including interpolated strings, interpolated string handlers, and String.Create
- String encoding including UTF-8 support

**Chapter 7**

- Detailed description of regions' impact on the GC process
- Using dotnet-gcmon to analyze induced GCs
- Explanation and impact of the SuppressGCTransition attribute
- Analyze suspension time with dotnet-gcmon
- Setting hard memory limits
- Provisional mode
- "Servo tuning"

**Chapter 8**

- How to remotely trigger a GC with dotnet-trace
- Generational aware analysis

**Chapter 9**

- Details of free space in GC
- Special region experimental feature

**Chapter 10**

- Undocumented AllocationTick info
- Sweep and regions

**Chapter 11**

- Advanced dotnet-trace command syntax with --providers
- Large page support
- More configuration knobs (GC heap hard limit, GC heap count, GC thread affinity, memory load threshold, GC conservative mode, dynamic adaptation mode, gen0 and gen1 max budget)

**Chapter 12**

- Better IDisposable/finalizer pattern implementation
- IAsyncDisposable
- Using dotnet-gcdump and Visual Studio during memory leak investigation

**Chapter 13**

- DependentHandle usage
- Performance details about different ways to implement a thread-safe counter
- More on ref
- Inline arrays

**Chapter 14**

- Improvements in Span<T>
- New methods in the Unsafe class

**Chapter 15**

- Many new GC APIs including memory limits
- Undocumented frozen segment APIs
- Extended coverage of ClrMD
- EventPipe communication with TraceEvent and NetCore Client

## First Edition Introduction

In computer science, memory has been always there – from the punch cards through magnetic tapes to the nowadays sophisticated DRAM chips. And it will be always there, probably in the form of sci-fi holographic chips or even much more amazing things that we are now not able to imagine. Of course, the memory was there not without a reason. It is well known that computer programs are said to be algorithms and data structures joined together. I like this sentence very much. Probably everyone has at least once heard about the *Algorithms + Data Structures = Programs* book written by Niklaus Wirth (Prentice Hall, 1976), where this great sentence was coined.

From the very beginning of the software engineering field, memory management was a topic known by its importance. From the first computer machines, engineers had to think about the storage of algorithms (program code) and data structures (program data). It was always important how and where those data are loaded and stored for later use.

In this aspect, software engineering and memory management have been always inherently related, as much as software engineering and algorithms are. And I believe it always will be like that. Memory is a limited resource, and it always will be. Hence, at some point or degree, memory will always be kept in the minds of future developers. If a resource is limited, there always can be some kind of bug or misuse that leads to starvation of this resource. Memory is not an exception here.

Having said that, there is for sure one thing that is constantly changing regarding memory management – the quantity. First developers, or we should name them engineers, were aware of every single bit of their programs. Then they had kilobytes of memory. From each and every decade, those numbers are growing and today we are living in times of gigabytes, while terabytes and petabytes are kindly knocking into the door waiting for their turn. As the memory size grows, the access times decrease, making it possible to process all this data in a satisfying time. But even though we can say memory is fast, simple memory-management algorithms that try to process all gigabytes of data without any optimizations and more sophisticated tunings would not be feasible. This is mostly because memory access times are improving slower than the processing power of CPUs utilizing them. Special care must be taken to not introduce bottlenecks of memory access, limiting the power of today's CPUs.

This makes memory management not only of crucial importance, but also a really fascinating part of computer science. Automatic memory management makes it even better. It is not as easy as saying "let the unused objects be freed." What, how, and when – those simple aspects of memory management make it continuously an ongoing process of improving the old and inventing new algorithms. Countless scientific papers and PhD theses are considering how to automatically manage memory in the most optimal way. Events like the International Symposium on Memory Management (ISMM) shows every year how much is done in this field, regarding garbage collection; dynamic allocation; and interactions with runtimes, compilers, and operating systems. And then academic research slightly changes into commercialized and open sourced products we use in everyday work.

.NET is a perfect example of a managed environment where all such sophistication is hidden underneath, available to developers as a pleasant, ready-to-use platform. And indeed, we can use it without any awareness of the underlying complexity, which is a great .NET achievement in general. However, the more performance aware our program is, the less possible it is to avoid gaining any knowledge about how and why things work underneath. Moreover, personally I believe it is just fun to know how things we use every day work!

I've written this book in a way that I would have loved to read many years ago – when I started my journey into the .NET performance and diagnostic area. Thus, this book does not start from a typical introduction about the heap and the stack or description of multiple generations. Instead, I start from the very fundamentals behind memory management in general. In other words, I've tried to write this book in a way that will let you sense this very interesting topic, not only showing "here is a .NET Garbage Collector and it does this and that." Providing information not only what, but also how, and more importantly – why – should truly help you understand what is behind the scene of .NET memory management. Hence, everything you will read in regard to this topic in the future should be more understandable to you. I try to enlighten you with knowledge a little more general than just related to .NET, especially in the first two chapters. This leads to deeper understanding of the topic, which quite often may be also applied to other software engineering tasks (thanks to an understanding of algorithms, data structures, and simply good engineering stuff).

I wanted to write this book in a manner pleasant for every .NET developer. No matter how experienced you are, you should find something interesting here. While we start from the basics, junior programmers quickly will have an opportunity to get deeper into .NET internals. More advanced programmers will find many implementation details more interesting. And above all, regardless of experience, everyone should be able to benefit from the presented practical examples of code and problem diagnoses.

Thus, knowledge from this book should help you to write better code – more performance and memory aware, utilizing related features without fear but with full understanding. This also leads to better performance and scalability of your applications – the more memory oriented your code is, the less exposed it is for resource bottlenecks and utilization of them not optimally. I hope you will find the "For Better Code, Performance, and Scalability" subtitle justified after reading this book.

I also hope all this makes this book more general and long lasting than just a simple description of the current state of the .NET framework and its internals. No matter how future .NET frameworks will evolve, I believe most of the knowledge in this book will be actually true for a long time. Even if some implementation details will change, you should be able to easily understand them because of the knowledge from this book. Just because underlying principles won't change so fast. I wish you a pleasant journey through the huge and entertaining topic of automatic memory management!

Having said that, I would like also to emphasize a few things that are not particularly present in this book. The subject of memory management, although it seems very specialized and narrow at the first glance, is surprisingly wide. While I touch a lot of topics, they are sometimes presented not as detailed as I would like, for lack of space. Even with such limitations, the book is around 1104 pages long! Those omitted topics include, for example, comprehensive references to other managed environments (like Java, Python, or Ruby). I also apologize to F# fans for so few references to this language. There were not enough pages for a solid description simply, and I did not want to publish anything not being comprehensive. I would also have liked to put much more attention to the Linux environment, but this is so fresh and uncovered by the tools

topic that at the time of writing, I only give you some proposals in Chapter 3 (and omitting the macOS world completely for the same reasons). Obviously, I've also omitted a large part of other, not directly memory-related part of performance in .NET – like multithreading topics.

Secondly, although I've done my best to present practical applications of the topics and techniques discussed, this is not always possible without doing so in a completely exhausting way. Practical applications are simply too many. I rather expect from a reader reading comprehensively, rethinking the topic, and applying the knowledge gained in their regular work. Understand how something works and you will be able to use it!

This especially includes so-called scenarios. Please note that all scenarios included in this book are for illustrative purposes. Their code has been distilled to the bare minimum to easier show the root cause of one single problem. There may be various other reasons behind the observed misbehaving (like many ways how managed memory leaks may be noticed). Scenarios were prepared in a way to help illustrate such problems with a single example cause as it is obviously not possible to include all probable reasons in a single book. Moreover, in real-world scenarios, your investigation will be cluttered with a lot of noisy data and false investigation paths. There is often no single way of solving the described issues and yet many ways how you can find the root cause during problems analysis. This makes such troubleshooting a mix of a pure engineering task with a little of an art backed by your intuition. Please note also that scenarios sometimes reference to each other to not repeat themselves again and again with the same steps, figures, and descriptions.

I especially refrained from mentioning various technology-specific cases and sources of problems in this book. They are simply… too much technology specific. If I was writing this book 10 years ago, I would probably have had to list various typical scenarios of memory leaks in ASP.NET WebForms and WinForms. A few years ago? ASP.NET MVC, WPF, WCF, WF,… Now? ASP.NET Core, EF Core, Azure Functions, what else? I hope you get the point. Such knowledge is becoming obsolete too soon. The book stuffed with examples of WCF memory leaks would hardly interest anyone today. I am a huge fan of saying: "Give a man a fish; you have fed him for today. Teach a man to fish; and you have fed him for a lifetime." Thus, all the knowledge in this book, all the scenarios, are teaching you how to fish. All problems, regardless of underlying specific technology, may be diagnosed in the same way, if enough knowledge and understanding are being applied.

All this also makes reading this book quite demanding, as it is sometimes full of details and maybe a little overwhelming amount of information. Despite everything, I encourage you to read in-depth and slow, resisting the temptation of only a skimming reading. For example, to take full advantage of this book, one should carefully study the code shown and presented figures (and not just look at them, stating that they are obvious, so they may be easily omitted).

We are living in a great time of open sourced CoreCLR runtime. This moves CLR runtime understanding possibilities to a whole new level. There is no guessing, no mysteries. Everything is in code, may be read, and understood. Thus, my investigations of how things work are heavily based on CoreCLR's code of its GC (which is shared with .NET Framework as well). I've spent countless days and weeks analyzing this huge amount of good engineering work. I think it is great, and I believe there are people who would also like to study famous gc.cpp file, with a size of several tens of thousands of lines of code. It has a very steep learning curve, however. To help you with that, I often leave some clues where to start CoreCLR code study with respect to described topics. Feel free to get an even deeper understanding from the gc.cpp points I suggest!

After reading this book you should be able to

- Write performance and memory-aware code in .NET. While presented examples are in C#, I believe with the understanding and toolbox you gain here, you will be able to apply this also to F# or VB.NET.
- Diagnose typical problems related to .NET memory management. As most techniques are based on ETW/LLTng data and SOS extension, they are applicable both on Windows and Linux (with much more advanced tooling available on Windows).

- Understand how CLR works in the memory management area. I've put quite a lot of attention to explain not only how things work but also why.
- Read with the full understanding of many interesting C# and CLR runtime issues on GitHub and even participate with your own thoughts.
- Read the code of the GC in CoreCLR (especially gc.cpp) file with enough understanding to make further investigations and studies.
- Read with the full understanding of information about GCs and memory management in different environments like Java, Python, or Go.

As to the content of the book itself, it presents as follows. Chapter 1 is a very general theoretical introduction to memory management, without almost any reference to .NET in particular. Chapter 2 is similarly a general introduction to memory management on the hardware and operating system level. Both chapters may be treated as an important, yet optional introduction. They give a helpful, broader look at the topic, useful in the rest of the book. While I obviously and strongly encourage you to read them, you may omit them if you are in a hurry or interested only in the most practical, .NET-related topics. A note to advanced readers – even if you think topics from those two first chapters are well known to you, please read them. I've tried to include there not only obvious information, which you may find interesting.

Chapter 3 is solely dedicated to measurements and various tools (among which some are very often used later in the book). It is a reading that contains mainly a list of tools and how to use them. If you are interested mostly in the theoretical part of the book, you may only skim through it briefly. On the other hand, if you plan to use the knowledge of this book intensively in the diagnosis of problems, you will probably come back to this chapter often.

Chapter 4 is the first one where we start talking about .NET intensively, while still in a general way allowing us to understand some relevant internals like .NET type system (including value type vs. reference type), string interning, or static data. If you are really in a hurry, you may wish to start reading from there. Chapter 5 described the first truly memory-related topic – how memory is organized in .NET applications, introducing the concept of Small and Large Object Heap, as well as segments. Chapter 6 is going further into memory-related internals, dedicated solely to allocating memory. Quite surprisingly, quite a big chapter may be dedicated to such a theoretically simple topic. An important and big part of this chapter is the description of various sources of allocations, in the context of avoiding them.

Chapters from 7 to 10 are core parts describing how the GC works in .NET, with practical examples and considerations resulting from such knowledge. To not overwhelm with too much information provided at the same time, those chapters are describing the simplest flavor of the GC – so-called Workstation Non-Concurrent one. On the other hand, Chapter 11 is dedicated to describing all other flavors with comprehensive considerations that one can choose. Chapter 12 concludes the GC part of the book, describing three important mechanisms: finalization, disposable objects, and weak references.

The three last chapters constitute the "advanced" part of the book, in the sense of explaining how things work beyond the core part of .NET memory management. Chapter 13 explains, for example, the topic of managed pointers and goes deeper into structs (including recently added ref structs). Chapter 14 puts a lot of attention to types and techniques gaining more and more popularity recently, like Span<T> and Memory<T> types. There is also a smart section dedicated to the not-so-well known topic of data-oriented design and a few words about incoming C# features (like nullable reference types and pipelines). Chapter 15, the last one, describes various ways how we can control and monitor the GC from code, including GC class API, CLR Hosting, or ClrMD library.

Most of the listings from this book are available at the accompanying GitHub repository at `https://github.com/Apress/pro-.net-memory`. It is organized into chapters and most of them contain two solutions: one for conducted benchmarks and one for other listings. Please note that while included projects contain listings, there is often more code for you to look at. If you want to use or experiment with a particular listing, the easiest way will be just to search for its number and play around with it and its usage. But I also encourage you to just look around in projects for particular topics for better understanding.

There are not so many important conventions I would like to mention here. The most relevant one is to differentiate two main concepts used throughout the rest of the book:

- Garbage collection (GC) – the generally understood process of reclaiming no-longer needed memory
- The Garbage Collector (the GC) – the specific mechanism realizing garbage collection, most obviously in the context of the .NET GC

This book is also pretty self-contained and does not refer to many other materials or books. Obviously, there is a lot of great knowledge out there, and I would need to refer to various sources many times. Instead, let me just list the suggested books and articles of my choice as a complementary source of knowledge:

- *Pro .NET Performance* book written by Sasha Goldshtein, Dima Zurbalev, and Ido Flatow (Apress, 2012)
- *CLR via C#* book written by Jeffrey Richter (Microsoft Press, 2012)
- *Writing High-Performance .NET Code* by Ben Watson (Ben Watson, 2014)
- *Advanced .NET Debugging* by Mario Hewardt (Addison-Wesley Professional, 2009)
- *.NET IL Assembler* by Serge Lidin (Microsoft Press, 2012)
- *Shared Source CLI Essentials* by David Stutz (O'Reilly Media, 2003)
- "Book Of The Runtime" open source documentation developed in parallel to the runtime itself, available at https://github.com/dotnet/coreclr/blob/master/Documentation/botr/README.md

There is also a huge amount of knowledge from various online blogs and articles. But instead of flooding those pages with a list of them, let me just redirect you to a great https://github.com/adamsitnik/awesome-dot-net-performance repository maintained by Adam Sitnik.

CHAPTER 1

# Basic Concepts

Let's start with a simple yet essential question. When should you care about .NET memory management if it is all automated? Should you care at all? As you probably expect by the fact that we wrote such a book, we strongly encourage you to think about memory in every developer's situation. This is often what makes the difference between proof of concept on a developer machine and production-ready code. Is it optimal in terms of CPU and memory usage? Is it maintainable, testable, opened for extension, but closed for modification? Is your code SOLID? We believe all those questions distinguish beginners from more advanced, experienced programmers. The former are mainly interested in getting the job done and do not care much about the abovementioned, nonfunctional aspects of their work. The latter are experienced enough to have enough "mental processing power" to consider the quality of their work. We believe everyone wants to be like that. But this is, of course, not a trivial task. Writing elegant code, without any bugs, with each possible nonfunctional requirement fulfilled is really hard.

But should such a desire for excellency be the only motivation for gaining in-depth knowledge about .NET memory management? Memory corruptions causing `AccessViolationException` are extremely rare.[1] The rare scenario of uncontrolled increase in memory usage can also trigger the same exception. Do you have anything to be worried about then? Thanks to the sophistication of the .NET runtime, you do not have to think about memory aspects a lot. But, on the other hand, when being involved in analyzing performance problems of large .NET-based applications, memory consumption problems were always high on the list of issues. Does it cause trouble in the long-term view if you have a memory leak after days of continuous running? On the Internet, you can find a funny meme about a memory leak that was not fixed in the software of some particular combat missile, because there was enough memory before the missile reached its destination. Is your system such a one-time missile? Maybe you could use only two servers instead of ten? And further, you have to think about memory consumption especially in the times of serverless cloud computing. One of the examples can be Azure Functions, which are billed based on a measure called "gigabyte seconds" (GB-s). It is calculated by multiplying the average memory size in gigabytes by the time in seconds it takes to execute a particular function. Memory consumption directly translates into spent money.

In each case, you begin to realize that you have no idea where to start looking for the real cause and meaningful measurements. This is the place where you begin to understand that it is worthwhile to understand the internal mechanisms of your applications and the underlying runtime.

To deeply understand memory management in .NET, it is best to start from scratch. No matter whether you are a novice programmer or a very advanced one. It is important that you go through the theoretical introduction in this chapter: this will establish a necessary level of knowledge and understanding of concepts, which will be used through the rest of the book. For this not to be simply boring theory, sometimes

[1] `AccessViolationException` or other heap corruption can often be triggered by the automatic memory management, not because it is the cause, but because it is the heaviest memory-related component in the environment. Thus, it has the biggest possibility to reveal any inconsistent memory states.

© Konrad Kokosa, Christophe Nasarre, and Kevin Gosse 2024
K. Kokosa et al., *Pro .NET Memory Management*, https://doi.org/10.1007/979-8-8688-0453-3_1